Ingeniería en Desarrollo de Software

Actividad | #3 |

Programa Banco Mexicano (parte 2)

Lenguajes de Programación IV

Ingeniería en Desarrollo de Software

TUTOR: Aarón Iván Salazar Macías

ALUMNO: Oscar Esteban Sánchez Leyva

FECHA: 04/Abril/2026

ÍNDICE

ÍNDICE 2

INTRODUCCIÓN 3

INVESTIGACIÓN (CLASES ABSTRACTAS) 4

JAVA 4

PYTHON 5

C# 6

C++ 7

PHP 8

JUSTIFICACIÓN 9

DESARROLLO 10

DISEÑO DE INTERFAZ JAVA 10

CONCLUSIÓN 15

REFERENCIAS 16

INTRODUCCIÓN

En el presente documento se busca avanzar en el desarrollo de un sistema bancario básico en Java, continuando la labor previa. En esta segunda fase, se amplían las funcionalidades del programa con nuevas opciones en un menú interactivo, como consultar saldo y salir, además de las funciones básicas de depósito y retiro ya implementadas.

El proyecto sigue el paradigma de programación orientada a objetos, que facilita la organización, mantenimiento y reutilización del código mediante el uso de clases, métodos y atributos para modelar un sistema bancario real en un entorno práctico.

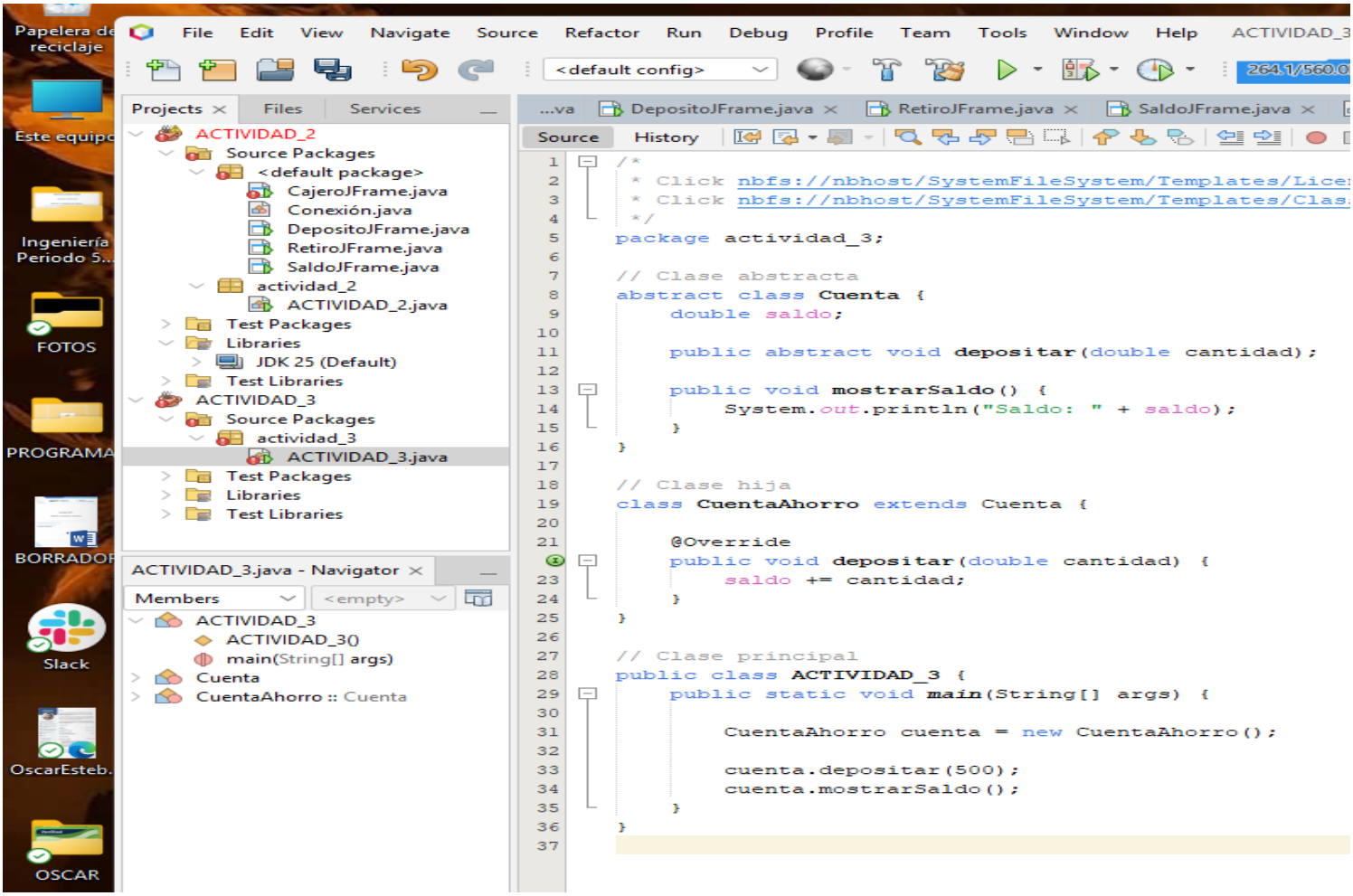
También se refuerza el empleo de estructuras de control como condicionales y ciclos para la toma de decisiones dentro del programa, asegurando la correcta validación de los datos ingresados por el usuario. Además, se pone énfasis en la interacción con el usuario a través del teclado, simulando un entorno más realista.

Paralelamente, se investiga el uso de clases abstractas en diferentes lenguajes de programación, lo que amplía el entendimiento sobre herencia y reutilización de código, destacando similitudes y diferencias entre tecnologías. Esto contribuye al fortalecimiento del conocimiento en desarrollo de software.

En conjunto, esta actividad consolida habilidades técnicas y fomenta el análisis y la resolución de problemas esenciales para el desarrollo profesional en programación.

INVESTIGACIÓN (CLASES ABSTRACTAS)

JAVA



Explicación:

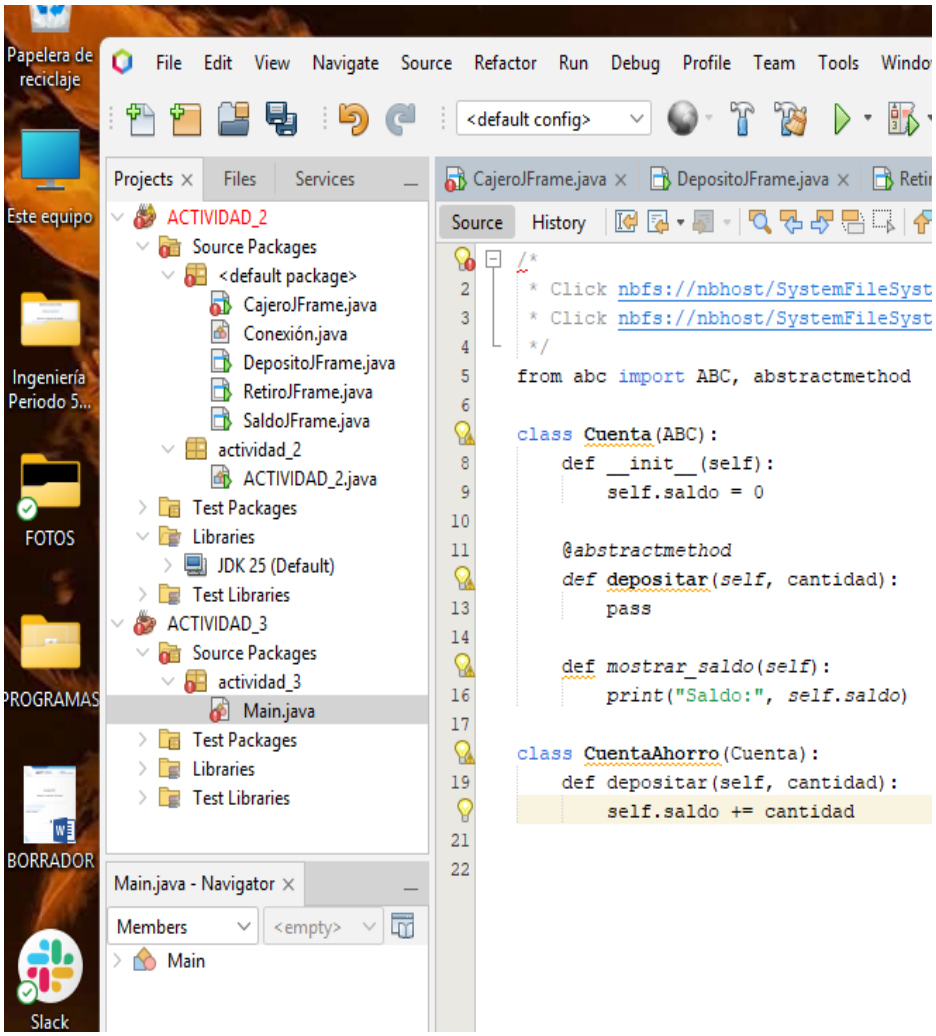
abstract class: Representa una clase base que no puede ser instanciada directamente y se utiliza como plantilla para que otras clases la extiendan.

abstract: Señala que un método carece de implementación, lo que obliga a las clases derivadas a proporcionarla.

extends: Facilita que una clase herede propiedades y métodos de otra, permitiendo la reutilización del código.

PYTHON

```java
// Clase abstracta
abstract class Cuenta {
    double saldo;

    // Constructor
    public Cuenta() {
        saldo = 0;
    }

    // Método abstracto
    public abstract void depositar(double cantidad);

    // Método normal
    public void mostrarSaldo() {
        System.out.println("Saldo: " + saldo);
    }
}

// Clase hija
class CuentaAhorro extends Cuenta {

    @Override
    public void depositar(double cantidad) {
        saldo += cantidad;
    }
}

// Clase principal
public class Main {
    public static void main(String[] args) {

        CuentaAhorro cuenta = new CuentaAhorro();

        cuenta.depositar(500);
        cuenta.mostrarSaldo();
    }
}
```

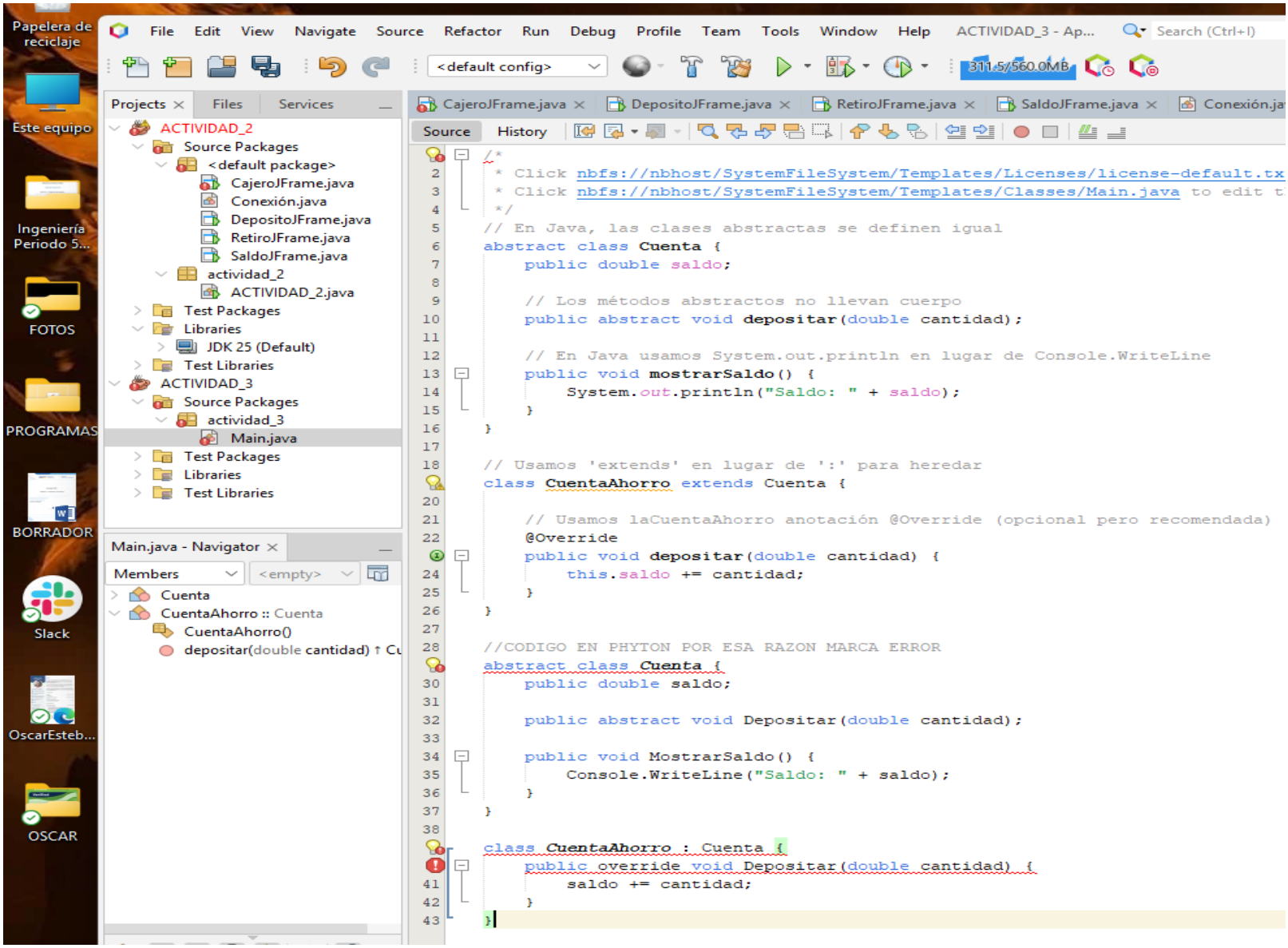


Explicación:

La notación **“ABC”** se emplea para declarar una clase abstracta en Python. Este tipo de clases sirve como base o plantilla para otras clases, pero no es posible instanciarla directamente.

El decorador **“@abstractmethod”** se utiliza para señalar que un método debe ser implementado obligatoriamente por las clases derivadas. En caso de que dicho método no sea implementado en la clase hija, se generará un error en tiempo de ejecución, lo cual garantiza que todas las subclases cumplan con la implementación requerida.

C#



Explicación:

<u>Abstract:</u> Sirve para representar una clase o un método que no está completamente definido. Una clase abstracta no puede ser instanciada, y los métodos declarados como abstractos deben ser obligatoriamente implementados por las clases hijas.

<u>@Override:</u> Señala que un método está sobrescribiendo o implementando uno que ha sido heredado de la clase padre.

C++

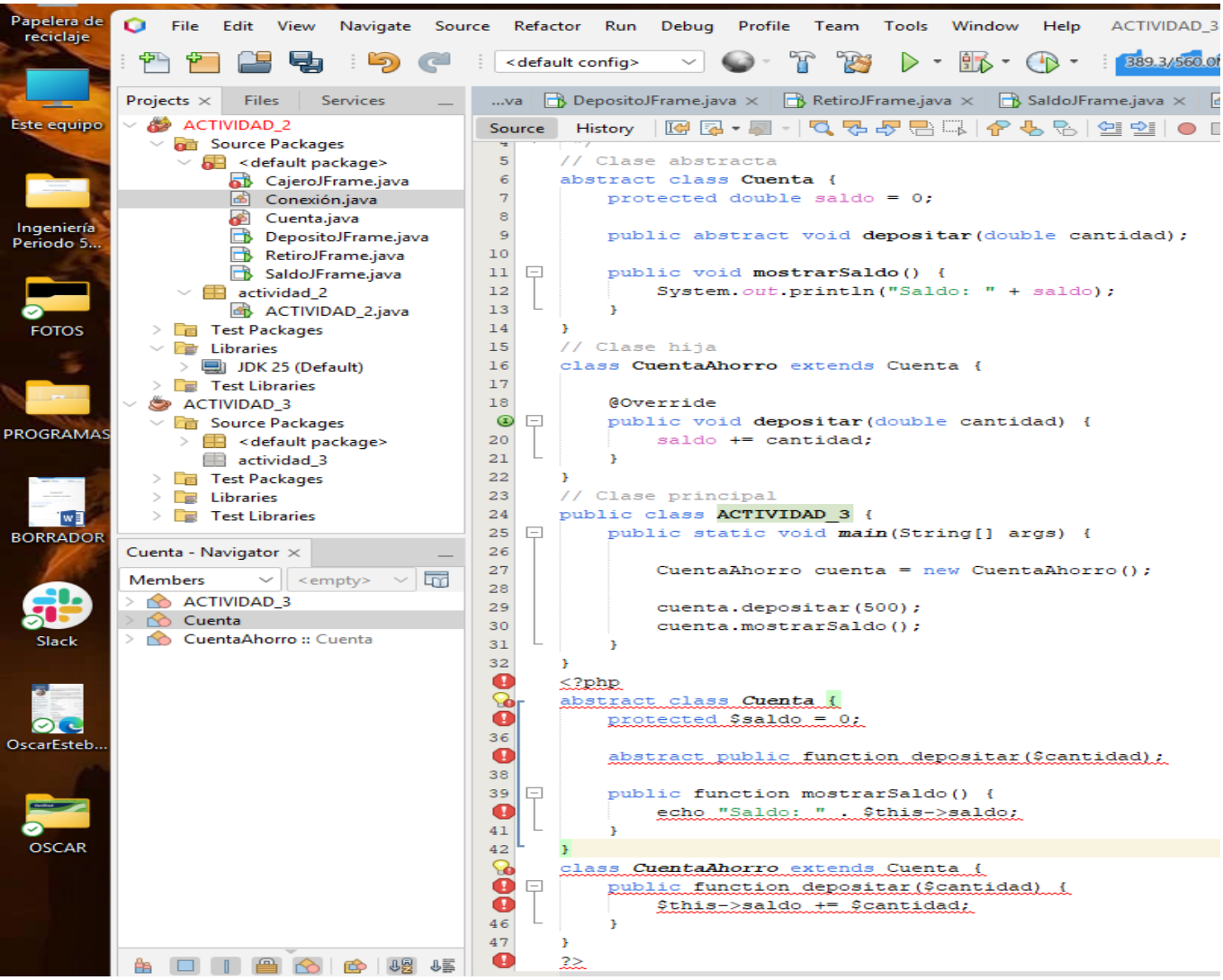
```java
*/
// En Java, las clases abstractas se definen con la palabra 'abstract'
abstract class Cuenta {
    // Usamos 'protected' para que las subclases puedan acceder al saldo direc
    protected double saldo;

    // Un método virtual puro en C++ equivale a un método 'abstract' en Java
    public abstract void depositar(double cantidad);

    public void mostrarSaldo() {
        System.out.println("Saldo: " + saldo);
    }
}

// 'extends' reemplaza a ': public' para la herencia
class CuentaAhorro extends Cuenta {

    // @Override es opcional pero buena práctica
    @Override
    public void depositar(double cantidad) {
        this.saldo += cantidad;
    }
}

// Ejemplo de uso (puedes poner esto en un archivo llamado Main.java)
public class Main {
    public static void main(String[] args) {
        CuentaAhorro miCuenta = new CuentaAhorro();
        miCuenta.depositar(500.0);
        miCuenta.mostrarSaldo();
    }
}
```

```cpp
/*
 * Click nbfs://nbhost/SystemFileSystem/Template
 * Click nbfs://nbhost/SystemFileSystem/Template
 */
// CODIGO DE C++ POR ESO MARCA ERROR
#include <iostream>
using namespace std;

class Cuenta {
public:
    double saldo;

    virtual void depositar(double cantidad) = 0;

    void mostrarSaldo() {
        cout << "Saldo: " << saldo << endl;
    }
};

class CuentaAhorro : public Cuenta {
public:
    void depositar(double cantidad) {
        saldo += cantidad;
    }
};
```

Explicación:
El operador "= 0" en C++ se utiliza para definir un método como abstracto o puro, lo cual implica que las clases derivadas estarán obligadas a implementar dicho método. Esta característica es fundamental en el diseño orientado a objetos, ya que asegura que las clases hijas adapten la funcionalidad específica según sus necesidades.

Por otro lado, el uso de la palabra clave "virtual" habilita la posibilidad de que un método sea sobrescrito en las clases derivadas. Esto resulta esencial para aprovechar las ventajas de la herencia y el polimorfismo, permitiendo que los objetos de una clase hija puedan comportarse adecuadamente al acceder a métodos que comparten la misma firma en la clase base.

PHP



Explicación:

abstract class: Se refiere a una clase base que actúa como modelo o plantilla y no puede ser instanciada directamente.

Extends: Es un mecanismo que posibilita que una clase herede propiedades y métodos de otra clase.

$this->: En PHP, se utiliza para acceder a los atributos o métodos pertenecientes al mismo objeto dentro de la clase.

JUSTIFICACIÓN

La presente actividad se justifica en la necesidad de fortalecer los conocimientos adquiridos en programación, haciendo énfasis en el lenguaje Java y el enfoque de la programación orientada a objetos. La creación de un sistema bancario básico brinda la oportunidad de aplicar de manera práctica conceptos esenciales como clases, métodos, estructuras de control y validación de datos, que son fundamentales en la formación de un desarrollador de software.

Además, la incorporación de funcionalidades como depósitos, retiros y consultas de saldo simula escenarios del mundo real, permitiendo comprender cómo los sistemas informáticos pueden abordar necesidades prácticas, en este caso, relacionadas con la gestión de operaciones financieras básicas.

Por otro lado, esta actividad es igualmente significativa porque promueve el desarrollo del pensamiento lógico y la capacidad de análisis, competencias clave para resolver problemas tecnológicos. Complementariamente, investigar sobre clases abstractas en diversos lenguajes de programación amplía la visión del estudiante, facilitándole el entendimiento de distintos enfoques para implementar herencia y reutilización de código.

En resumen, esta actividad no solo tiene un profundo valor académico, sino también un impacto práctico, ya que sienta las bases para el diseño de sistemas más avanzados y contribuye al desarrollo integral en el campo de la ingeniería de software.

DESARROLLO

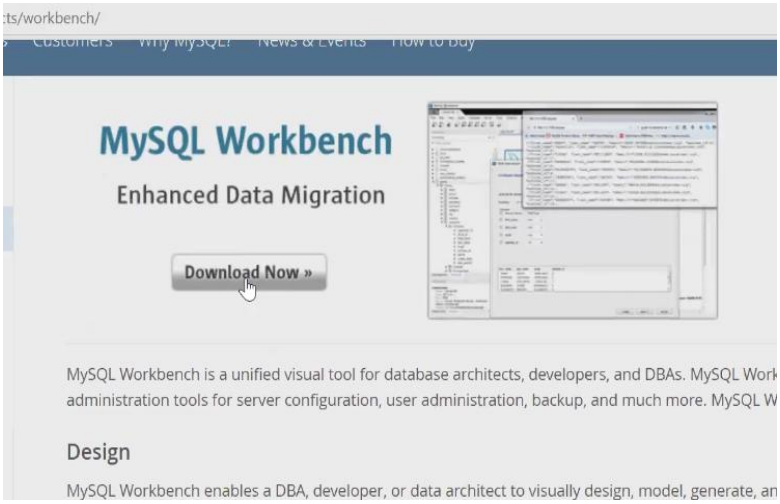
DISEÑO DE INTERFAZ JAVA

En primer lugar, procederé a describir de manera resumida el desarrollo de la actividad realizada. A continuación, adjuntaré la evidencia correspondiente. El programa desarrollado consiste en un sistema bancario básico implementado en el lenguaje de programación Java. Este sistema permite al usuario llevar a cabo diversas operaciones comunes, tales como realizar depósitos, efectuar retiros, consultar el saldo de su cuenta y finalizar la sesión en el sistema.

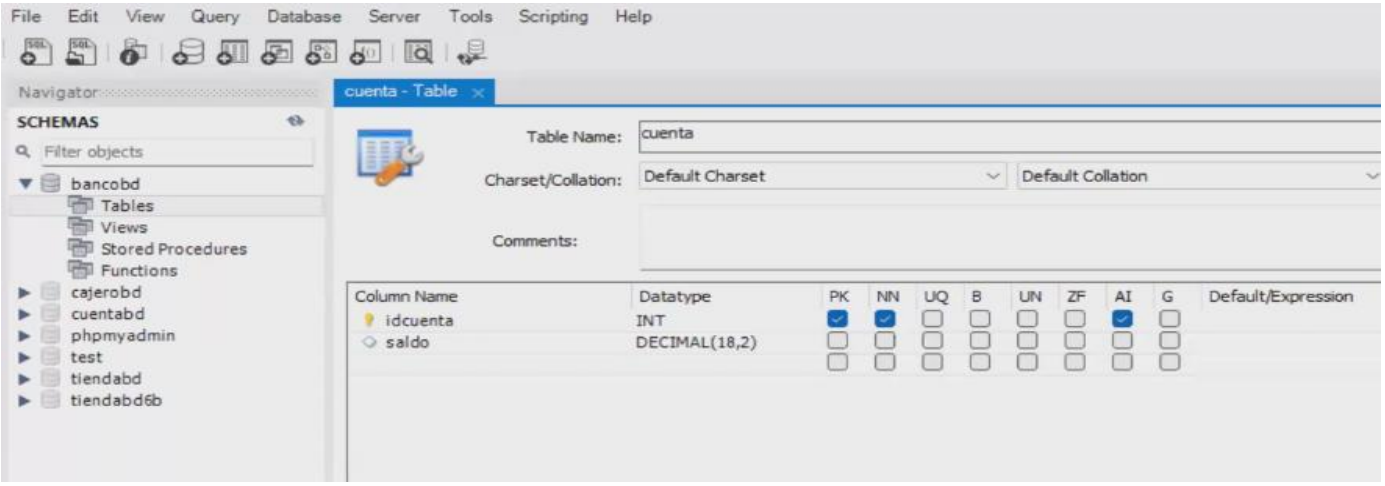
Para la implementación, se empleó el paradigma de la programación orientada a objetos, a través del diseño y creación de la clase CuentaBancaria. Esta clase fue diseñada para encapsular el atributo correspondiente al saldo de la cuenta, así como para integrar los métodos pertinentes que gestionan las transacciones y las interacciones necesarias con el usuario.

La funcionalidad del sistema se organiza mediante un menú interactivo, construido para solicitar al usuario que seleccione una opción utilizando su teclado. Según la opción elegida por el usuario, se ejecuta automáticamente una acción específica gracias al empleo de una estructura de control basada en un switch.

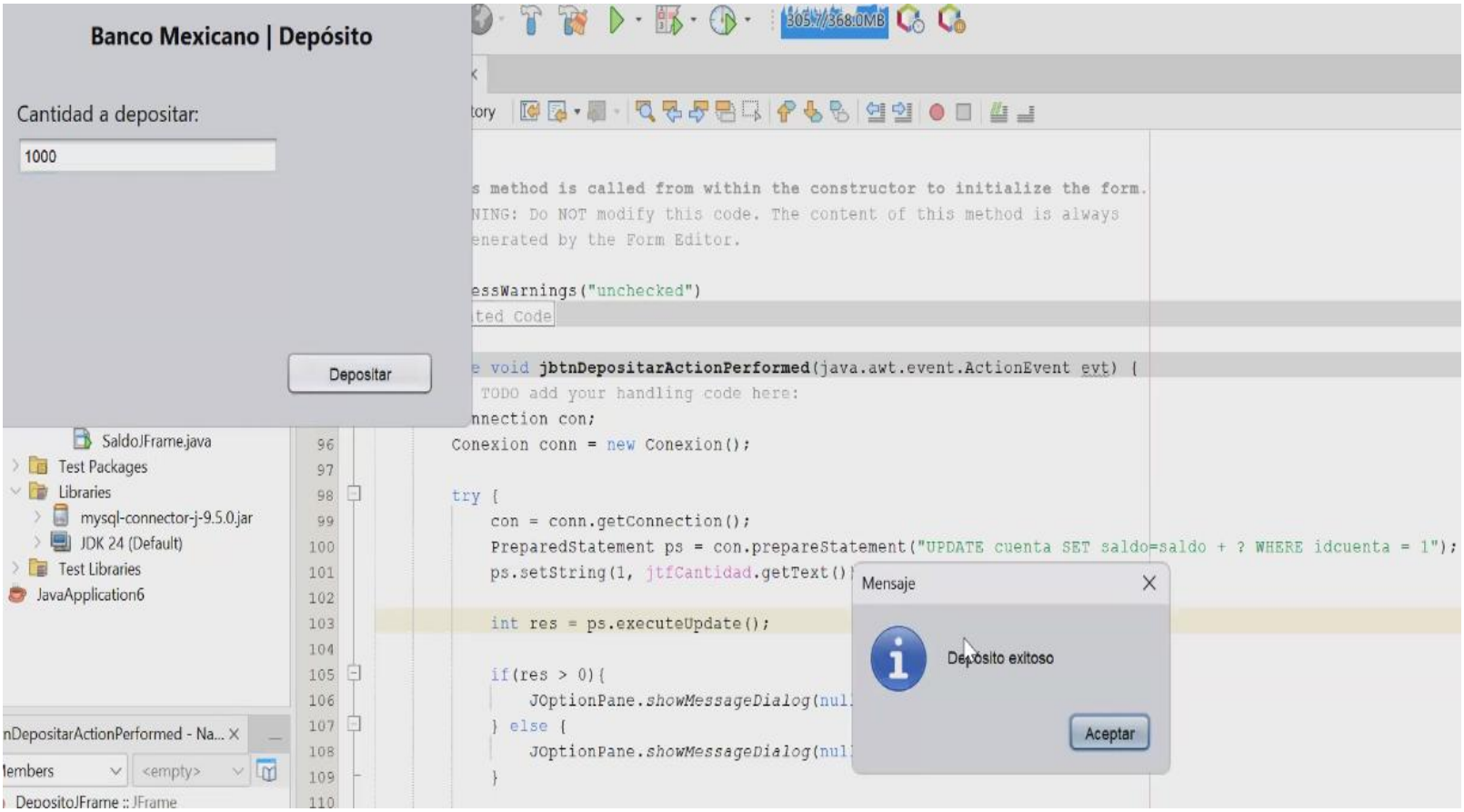
Adicionalmente, para garantizar que se controle correctamente el flujo de las operaciones y se eviten errores comunes, se implementaron diversas validaciones utilizando estructuras condicionales del tipo if. Estas validaciones aseguran, por ejemplo, que no se permita realizar depósitos de valores negativos o intentos de retiro cuando los fondos disponibles en la cuenta resulten insuficientes para cubrir la operación solicitada.

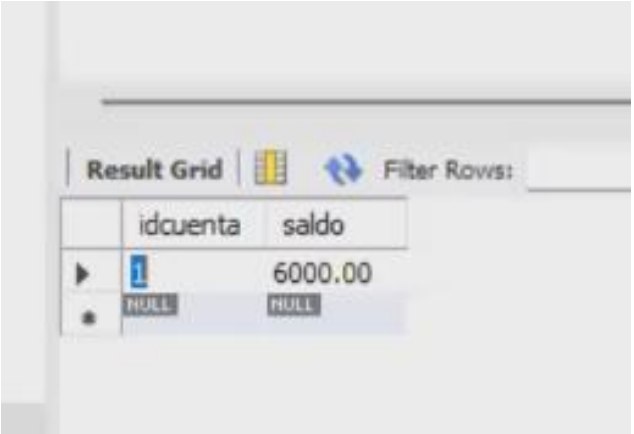


```java
import java.sql.Connection;
import java.sql.DriverManager;

/**
 *
 * @author Aarón I. Salazar
 */
public class Conexion {

    public Connection getConnection(){
        Connection con = null;
        String base = "bancobd";
        String url = "jdbc:mysql://localhost:3306/" + base;
        String user = "root";
        String password = "";

        try{
            Class.forName("com.mysql.jdbc.Driver");
            con = (Connection) DriverManager.getConnection(url, user, password);
        } catch(Exception e){
            System.err.println(e);
        }
        return con;
    }
}
```

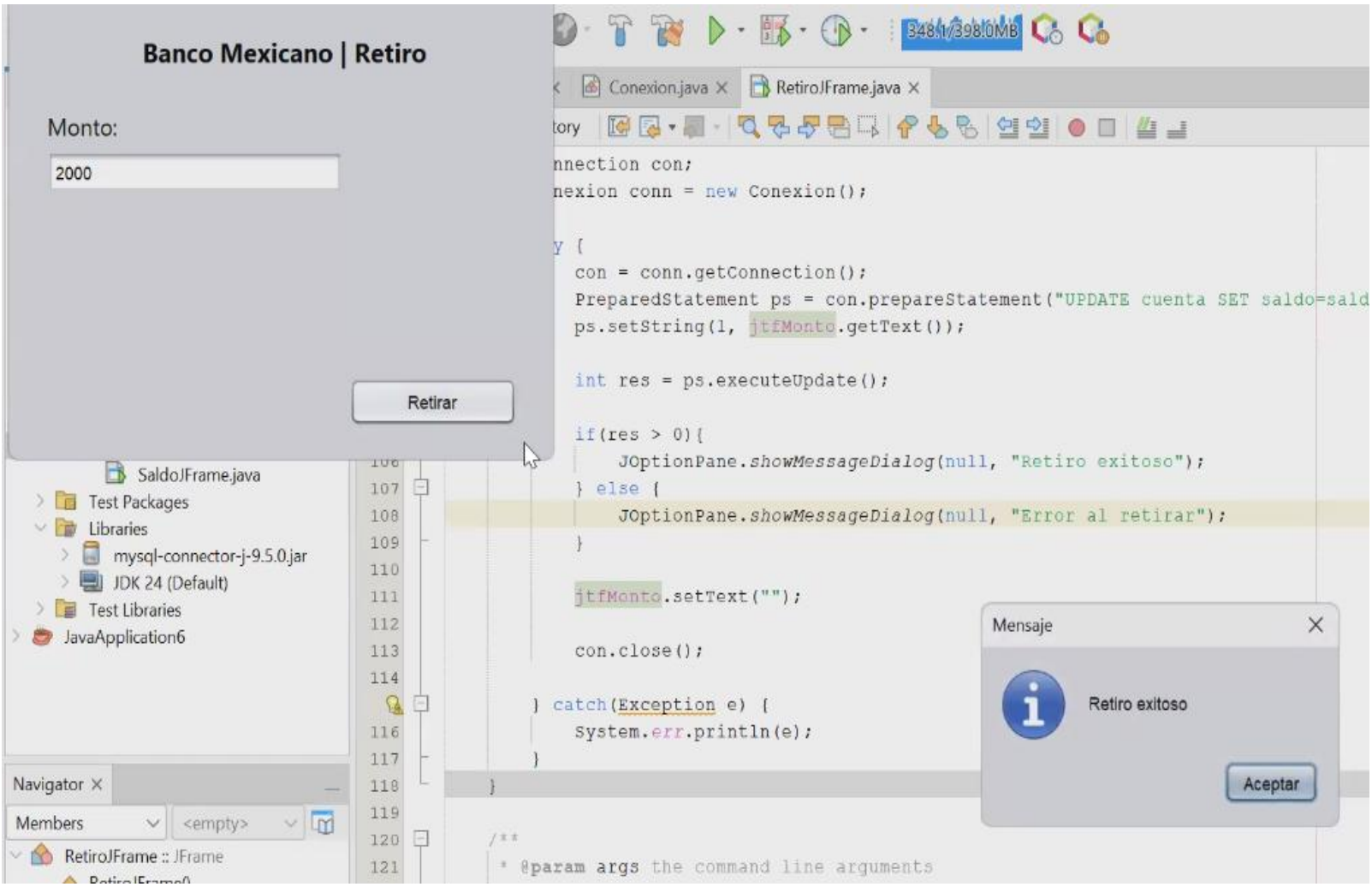



```java
private void jbtnDepositarActionPerformed(java.awt.event.ActionEvent evt) {
    // TODO add your handling code here:
    Connection con;
    Conexion conn = new Conexion();

    try {
        con = conn.getConnection();
        PreparedStatement ps = con.prepareStatement("UPDATE cuenta SET saldo=saldo + ? WHERE id = 1");
        ps.setString(1, jtfCantidad.getText());

        int res = ps.executeUpdate();

        if(res > 0){
            JOptionPane.showMessageDialog(null, "Depósito exitoso");
        } else {
            JOptionPane.showMessageDialog(null, "Error al depositar");
        }

        jtfCantidad.setText("");

        con.close();

    } catch(Exception e) {
        System.err.println(e);
```





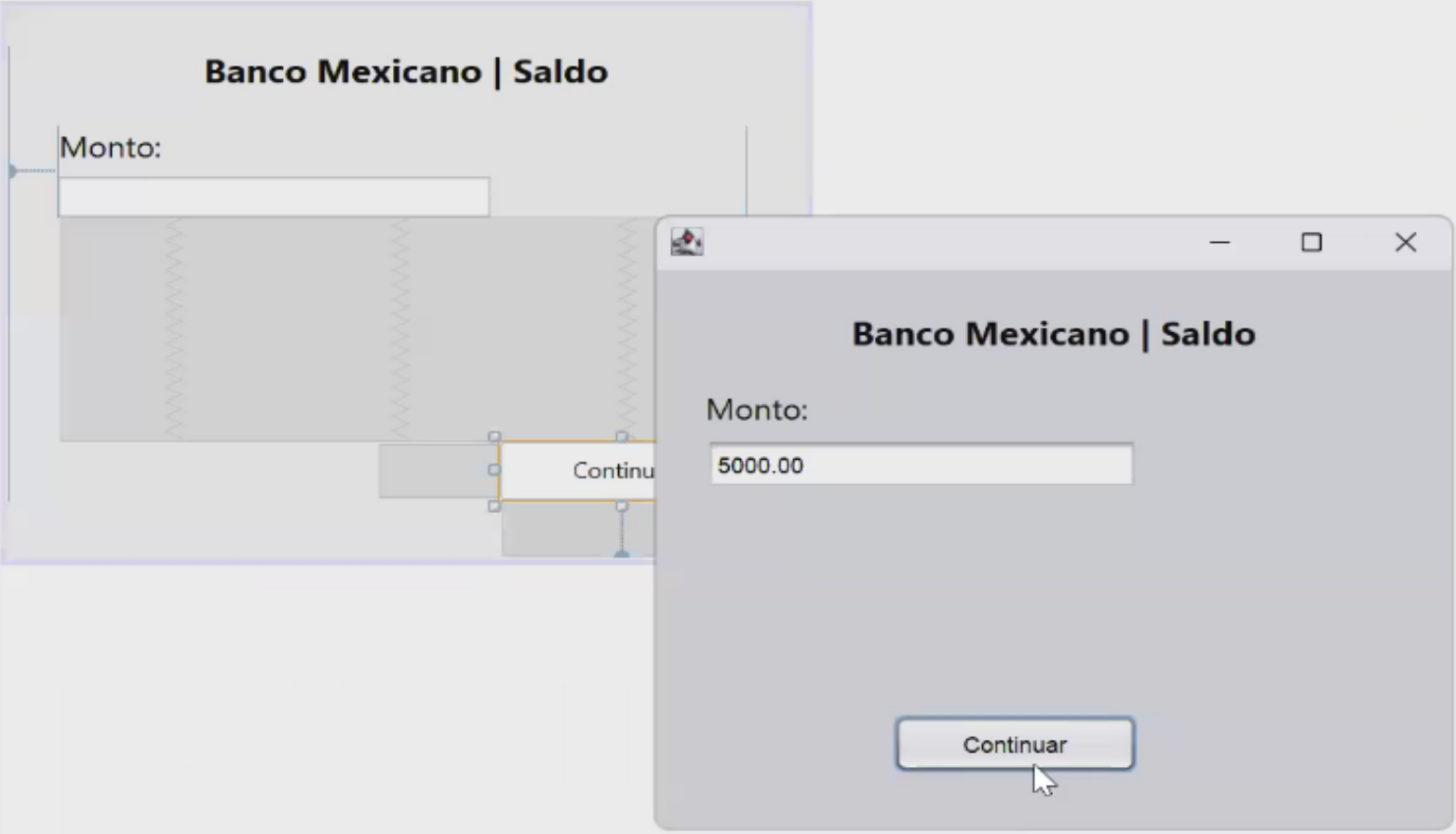

```java
try {
    con = conn.getConnection();
    PreparedStatement ps = con.prepareStatement("UPDATE cuenta SET saldo=saldo - ? WHERE idcuenta = 1");
    ps.setString(1, jtfMonto.getText());

    int res = ps.executeUpdate();

    if(res > 0){
        JOptionPane.showMessageDialog(null, "Retiro exitoso");
    } else {
        JOptionPane.showMessageDialog(null, "Error al retirar");
    }

    jtfMonto.setText("");

    con.close();

} catch(Exception e) {
    System.err.println(e);
}
}
```



Result Grid | Filter Rou

idcuenta	saldo
1	5000.00
NULL	

```
try {
    con = conn.getConnection();
    PreparedStatement ps = con.prepareStatement("SELECT saldo FROM cuenta WHERE idcuenta = 1");

    ResultSet rs = ps.executeQuery();

    if(rs.next()){
        jtfMonto.setText(rs.getString("saldo"));
    } else {
        JOptionPane.showMessageDialog(null, "Error");
    }

    con.close();

} catch(Exception e) {
    System.err.println(e);
}
}
```



CONCLUSIÓN

En esta actividad se diseñó y desarrolló un sistema bancario básico en Java que integró tanto la lógica del programa como una interfaz funcional para que el usuario pudiera interactuar mediante un menú. Esto facilitó la práctica de los conceptos clave de la programación orientada a objetos, como la creación de clases, el uso de métodos y la aplicación del principio de encapsulación.

Se trabajó también en el manejo de estructuras de control, como condicionales y ciclos, las cuales resultaron fundamentales para implementar la lógica de decisiones en el programa. Asimismo, se incorporaron validaciones que aseguraron la correcta funcionalidad del sistema, previniendo errores durante las operaciones realizadas por los usuarios.

Además, se llevó a cabo una investigación acerca de clases abstractas en diferentes lenguajes de programación, lo que profundizó en conceptos como la herencia y la reutilización de código. Este análisis permitió comprender mejor las similitudes y diferencias entre diversos entornos de desarrollo.

En conclusión, la realización de esta actividad supuso un importante avance en la consolidación de habilidades prácticas en programación. Permitió fortalecer la capacidad de análisis, la lógica y la resolución de problemas, estableciendo una base sólida para abordar proyectos de mayor complejidad en el futuro.

REFERENCIAS

Qué es una clase abstracta en la programación orientada a objetos. (s. f.). https://platzi.com. https://platzi.com/blog/clases-abstractas/

Llamas, L. (2024, 23 noviembre). *Qué son y cómo usar las clases abstractas*. Luis Llamas. https://www.luisllamas.es/que-es-una-clase-abstracta-en-programacion/

Ordoñez, J. O. (s. f.). *Clases y Miembros Abstractos en C#*. https://ortizol.blogspot.com/2013/12/clases-y-miembros-abstractos-en-c.html

atareao. (2023, 7 marzo). *Clases abstractas, interfaces y polimorfismo. Pyldoras Pythonicas.* [Vídeo]. YouTube. https://www.youtube.com/watch?v=l-HMdiw2SNM

Guinxu. (2017, 22 noviembre). *Guía para programar en CUALQUIER lenguaje | Parte 1/2 (C, C++, Java, Python, PHP, JavaScript, C#. . .)* [Vídeo]. YouTube. https://www.youtube.com/watch?v=Laqz6W2ONBI

Guinxu. (2017b, diciembre 7). *Guía para programar en CUALQUIER lenguaje | Parte 2/2 (C, C++, Java, Python, PHP, JavaScript, C#. . .)* [Vídeo]. YouTube. https://www.youtube.com/watch?v=DXtA3T9EHuY